

Benchmarking NIST-Standardised ML-KEM and ML-DSA on ARM Cortex-M0+: Performance, Memory, and Energy on the RP2040

Rojin Chhetri
rojinchhetri07@gmail.com

Abstract—The migration to post-quantum cryptography is urgent for Internet of Things devices with 10–20 year lifespans, yet no systematic benchmarks exist for the finalised NIST standards on the most constrained 32-bit processor class. This paper presents the first isolated algorithm-level benchmarks of ML-KEM (FIPS 203) and ML-DSA (FIPS 204) on ARM Cortex-M0+, measured on the RP2040 (Raspberry Pi Pico) at 133 MHz with 264 KB SRAM. Using PQClean reference C implementations, we measure all three security levels of ML-KEM (512/768/1024) and ML-DSA (44/65/87) across key generation, encapsulation/signing, and decapsulation/verification. ML-KEM-512 completes a full key exchange in 35.7 ms with an estimated energy cost of 2.83 mJ (datasheet power model)—17× faster than a complete ECDHP-256 key agreement on the same hardware. ML-DSA signing exhibits high latency variance due to rejection sampling (coefficient of variation 66–73%, 99th-percentile up to 1,125 ms for ML-DSA-87). The M0+ incurs only a 1.8–1.9× slowdown relative to published Cortex-M4 reference C results (compiled with `-O3` versus our `-Os`), despite lacking 64-bit multiply, DSP, and SIMD instructions—making this a conservative upper bound on the microarchitectural penalty. All code, data, and scripts are released as an open-source benchmark suite for reproducibility.

Index Terms—Post-quantum cryptography, ML-KEM, ML-DSA, FIPS 203, FIPS 204, ARM Cortex-M0+, RP2040, IoT security, lattice-based cryptography, benchmarking.

I. INTRODUCTION

Classical public-key algorithms—RSA and elliptic-curve cryptography (ECC)—will be rendered insecure by cryptographically relevant quantum computers [1]. The “harvest now, decrypt later” threat model means that data transmitted today must already be protected against future quantum adversaries, a concern amplified by the billions of IoT devices with operational lifespans of 10–20 years deployed in critical infrastructure such as power grids, medical systems, and industrial control networks [5].

In August 2024, the U.S. National Institute of Standards and Technology (NIST) published three post-quantum cryptography (PQC) standards: FIPS 203 (ML-KEM, a lattice-based key-encapsulation mechanism) [2], FIPS 204 (ML-DSA, a lattice-based digital signature algorithm) [3], and FIPS 205 (SLH-DSA, a hash-based signature scheme) [4]. NIST subsequently published FIPS 206 (FN-DSA, a lattice-based signature scheme derived from FALCON) in 2025; we leave its evaluation on M0+ to future work. NIST mandates that U.S. federal systems migrate to these standards by 2035, and the EU

Cyber Resilience Act similarly requires security updates for IoT devices sold in Europe.

The ARM Cortex-M4 processor—the target of the widely cited pqm4 benchmarking framework [7], [8]—is well characterised for PQC workloads. However, the even more constrained ARM Cortex-M0+ powers a vast population of Class-1 IoT devices (RFC 7228 [6]): smart meters, medical sensors, and industrial field nodes, typically costing under \$5 per unit. The Cortex-M0+ implements the ARMv6-M instruction set with approximately 52 instructions, no 64-bit multiply (`UMULL/SMULL`), no DSP or SIMD extensions, and only eight directly accessible low registers (R0–R7). Despite five prior studies exploring PQC on Cortex-M0 or M0+ hardware [9]–[13], none benchmarked the *finalised* FIPS 203 and 204 standards; all used pre-standardisation algorithm versions (Kyber, Dilithium, or Saber). This paper focuses on the two lattice-based standards (ML-KEM and ML-DSA); the hash-based SLH-DSA (FIPS 205) has fundamentally different performance characteristics—large signatures and deterministic but slow signing—warranting a separate study.

Contributions. This paper fills this gap with four specific contributions:

- 1) The first benchmarks of NIST-standardised ML-KEM (FIPS 203) and ML-DSA (FIPS 204) on Cortex-M0+, providing isolated algorithm-level timing, memory, and energy metrics across all security levels.
- 2) A variance analysis of ML-DSA signing latency characterising the impact of FIPS 204 rejection sampling on constrained hardware (100 iterations per security level, reporting mean, standard deviation, coefficient of variation, and 95th/99th percentiles).
- 3) A complete memory compatibility map (peak stack and flash footprint) for all six algorithm variants on a 264 KB SRAM device, determining feasibility for Class-1 IoT.
- 4) An open-source, reproducible benchmark suite—code, serial capture scripts, and raw data—enabling the community to reproduce or extend results on RP2040 hardware.

The remainder of the paper is structured as follows. Section II provides background on ML-KEM, ML-DSA, and the Cortex-M0+ architecture. Section III surveys related work. Sections IV and V describe the experimental methodology and present results. Section VI discusses practical implications, and Section VII concludes.

II. BACKGROUND

A. ML-KEM (FIPS 203)

ML-KEM is a lattice-based key-encapsulation mechanism derived from CRYSTALS-Kyber. It provides IND-CCA2 security through the module-learning-with-errors (Module-LWE) problem. FIPS 203 defines three parameter sets—ML-KEM-512, 768, and 1024—offering roughly 128, 192, and 256 bits of classical security, respectively [2]. Public keys range from 800 to 1,568 bytes, and ciphertexts from 768 to 1,568 bytes. All variants produce a 32-byte shared secret. The core computational kernel is the Number Theoretic Transform (NTT) over the polynomial ring $\mathbb{Z}_q[x]/(x^{256} + 1)$ with $q = 3329$.

B. ML-DSA (FIPS 204)

ML-DSA is a lattice-based digital signature scheme derived from CRYSTALS-Dilithium, providing EUF-CMA security via the Module-LWE and Module-SIS problems. FIPS 204 defines three parameter sets—ML-DSA-44, 65, and 87—at security levels roughly equivalent to AES-128, AES-192, and AES-256, respectively [3]. A critical characteristic of ML-DSA is its use of the *Fiat–Shamir with Aborts* paradigm: signing involves rejection sampling, where the signer generates a candidate signature and discards it if a norm check fails. The expected number of iterations is 4.25, 5.1, and 3.85 for levels 44, 65, and 87, respectively [3]. This makes signing time inherently non-deterministic—a key concern for real-time IoT applications.

C. ARM Cortex-M0+ and the RP2040

The ARM Cortex-M0+ implements the ARMv6-M architecture, the most constrained 32-bit ARM profile. Compared to the ARMv7E-M (Cortex-M4) used in the pqm4 framework, the Cortex-M0+ lacks three capabilities critical for PQC performance:

- **No 64-bit multiply result:** the `UMULL` and `SMULL` instructions that produce a 64-bit product from two 32-bit operands are absent. Montgomery and Barrett reduction steps must be decomposed into sequences of 32-bit multiplies [12].
- **No DSP/SIMD:** the Cortex-M4 can process two 16-bit polynomial coefficients simultaneously using halfword SIMD instructions; the Cortex-M0+ cannot.
- **Limited register file:** only eight low registers (R0–R7) are directly accessible with the Thumb ISA, versus the full 16-register set on Cortex-M4. No barrel shifter is available.

The RP2040 (Raspberry Pi Pico) integrates a dual-core Cortex-M0+ at 133 MHz with 264 KB SRAM and 2 MB QSPI flash. Critically, the RP2040 implements the *single-cycle* $32 \times 32 \rightarrow 32$ -bit multiplier option—many Cortex-M0+ implementations use a 32-cycle multiplier instead. This design choice is what makes PQC computationally feasible on this platform [23].

III. RELATED WORK

A. PQC on Cortex-M4

The pqm4 project [7], [8] is the de facto benchmarking standard for PQC on ARM microcontrollers, providing cycle counts for ML-KEM, ML-DSA, SLH-DSA, and additional NIST candidates on the STM32F4 (Cortex-M4). Subsequent work has extended M4 benchmarks to Cortex-M4/M7 porting via the SLOTHY optimiser [15], efficient sparse polynomial multiplication for ML-DSA [16], and ML-DSA on 16-bit MSP430 processors [17]. The pqmx project [18] targets Cortex-M55/M85 with Helium MVE extensions—skipping the Cortex-M0+ entirely. No “pqm0+” equivalent exists.

B. Prior M0+/M0 Work

Five prior works have explored PQC on Cortex-M0 or M0+ hardware. We summarise each and explain the gap our work fills.

Halak et al. [9] benchmarked pre-standardisation Kyber-512 and Dilithium-2 on the Raspberry Pi Pico W within TLS handshakes using MbedTLS with experimental liboqs integration (IEEE Access, 2024). They measured TLS-level latency, energy, and computation costs but did not isolate algorithm-level cycle counts, did not test all security levels, did not profile stack or flash usage, and used pre-standardisation algorithm versions.

Karmakar et al. [10] implemented Saber KEM on a Cortex-M0 with a memory-efficient design achieving 4.8–7.5 million cycles (TCHES, 2018). This proved lattice-based KEMs can fit on M0-class devices, but Saber was not selected by NIST for standardisation.

Bos, Gourjon, Renes et al. [11] implemented the first complete masked (side-channel-protected) Kyber decapsulation on an NXP FRDM-KL82Z (Cortex-M0+) and validated with 100,000 power traces (TCHES, 2021). Their focus was on side-channel protection overhead, not raw performance benchmarking, and they used pre-standardisation Kyber without ML-DSA.

Li, Wang & Wang [12] optimised NTT-based polynomial multiplication for Kyber and Saber on Cortex-M0/M0+ using hybrid k-reduction and multi-moduli NTTs, achieving a $\sim 2.9\times$ speedup for Saber’s polynomial multiplication (INDOCRYPT, 2023), building on the foundational NTT work by Chung et al. [28] for M0/M0+. They benchmarked only the polynomial multiplication component, not complete KEM or signature operations.

Bos, Renes & Sprenkels [13] created a compact Dilithium implementation requiring <7 KB for signing and <3 KB for verification, targeting M0+-class memory constraints (AFRICACRYPT, 2022). Despite targeting M0+ memory profiles, they benchmarked only on Cortex-M4 (pqm4) and used pre-standardisation Dilithium.

Summary. All five prior M0+/M0 studies used pre-standardisation algorithm versions (Kyber, Dilithium, or Saber). None provide isolated, algorithm-level benchmarks of the finalised FIPS 203 and 204 standards with stack, flash, and energy metrics across all security levels. Table I summarises the key differences.

TABLE I
COMPARISON WITH PRIOR M0+/M0 PQC WORK. “STD.” INDICATES WHETHER THE FINALISED FIPS 203/204 STANDARDS WERE BENCHMARKED.

Work	Year	Hardware	Algorithm	Std.	Isolated metrics	All levels
Halak et al. [9]	2024	RP2040 (M0+)	Kyber/Dilithium	No	No (TLS-level)	No
Karmakar et al. [10]	2018	Cortex-M0	Saber	No	Yes	N/A
Bos et al. [11]	2021	NXP KL82Z (M0+)	Masked Kyber	No	Yes	No
Li et al. [12]	2023	Cortex-M0/M0+	Kyber/Saber poly. mul.	No	Partial	No
Bos et al. [13]	2022	Cortex-M4 (M0+ target)	Dilithium	No	Yes	No
This work	2026	RP2040 (M0+)	ML-KEM / ML-DSA	Yes	Yes	Yes

C. Broader PQC Landscape

Several recent works study PQC on IoT but do not target the M0+ processor class. Lopez et al. [19] evaluated PQC on Raspberry Pi 3B+ and 5 (Cortex-A, 64-bit Linux)—a fundamentally different class from bare-metal M0+. Grassl and Sturm [20] benchmarked PQC on Raspberry Pi models 1B through 4B, all running Cortex-A processors. Tschofenig et al. [21] studied PQC for over-the-air firmware updates on Cortex-M class devices but provided no M0+-specific cycle counts. Liu, Ramachandran et al. [5] provide a comprehensive survey of PQC for IoT with 86 citations but identify M0+-class benchmarking as an open problem. The Schwabe MSR talk [22] explicitly identified Cortex-M0 as a research target in 2019; as of 2026, this gap remains unfilled for standardised algorithms. On the commercial side, PQShield’s PQMicroLib-Core [30] achieves ML-KEM in just 5 KB RAM on Cortex-M, demonstrating that highly optimised implementations can reduce memory pressure significantly below our reference C measurements. Dinu [31] analysed the migration path from ECDSA to ML-DSA, providing additional context for the classical versus post-quantum comparison in our results.

IV. EXPERIMENTAL METHODOLOGY

A. Hardware Platform

All experiments were conducted on a Raspberry Pi Pico H (RP2040): dual-core ARM Cortex-M0+ at 133 MHz, 264 KB SRAM, 2 MB QSPI flash, with the single-cycle $32 \times 32 \rightarrow 32$ -bit multiplier [23]. Only one core was used for benchmarking; the second core remained idle.

B. Software Stack

PQC implementations. We used the PQCclean [24] reference C implementations of ML-KEM and ML-DSA, compiled with `-mcpu=cortex-m0plus -mthumb -Os` (optimise for size, the Pico SDK default) using `arm-none-eabi-gcc 12.2.1`. PQCclean provides portable, well-tested, pure C reference code without platform-specific assembly optimisations, forming the basis of the `pqm4` framework’s reference implementations. We deliberately chose reference C to establish a *baseline* for Cortex-M0+ capability; optimised assembly implementations would improve these numbers but are beyond the scope of this study. We note that the PQCclean repository is transitioning to read-only status (scheduled July 2026) as the community migrates to the PQCA `mlkem-native` and `mldsa-native` packages [32].

Our results were obtained from PQCclean at commit `3730b32a` (pinned prior to this transition); the reference C implementations are algorithmically identical to the PQCA successors (`mlkem-native` and `mldsa-native`), which derive from the same NIST-submitted code. The exact commit hash and build instructions are provided in the reproducibility repository to ensure long-term replicability.

The RP2040’s ring oscillator (ROSC) was used for random number generation via a thin `randombytes_pico.c` adapter. While the ROSC is adequate for benchmarking purposes—it does not affect execution timing since ML-KEM and ML-DSA call `randombytes()` a fixed number of times per operation—it is not cryptographically certified. Production deployments would require a DRBG seeded from a certified entropy source (e.g., NIST SP 800-90A).

Classical baselines. RSA-2048, ECDSA P-256, and ECDH P-256 were benchmarked using `mbedtlsTLS 3.6.0`, bundled with the Raspberry Pi Pico SDK v2.2.0 as the `pico_mbedtls` library. Our custom `mbedtls_config.h` (included in the repository) enables only the required algorithms—RSA PKCS#1 v1.5, ECDSA, ECDH on `secp256r1`—with no hardware-accelerated big-number operations, providing a direct comparison on identical hardware. Note that PQCclean does not include classical algorithms; the classical baselines are entirely from `mbedtls`.

C. Timing Methodology

Execution time was measured using the Pico SDK’s `time_us_32()` function, which provides $1\mu\text{s}$ resolution from the RP2040’s hardware timer. Each deterministic operation (key generation, encapsulation, decapsulation, verification) was executed 30 times; ML-DSA signing was executed 100 times per security level to capture rejection-sampling variance. No explicit warm-up iterations were performed. The Cortex-M0+ core itself lacks instruction and data caches, branch prediction, and speculative execution; the only caching layer is the RP2040’s 16 KB two-way set-associative XIP flash cache, which reaches steady state within the first few function calls. The consistently low CV ($<1.5\%$ for deterministic operations) confirms that first-iteration cache-cold effects are negligible across our 30-run samples. All measurements were performed on real hardware—no simulators or emulators were used. Correctness is verified at runtime: for ML-KEM, the encapsulated and decapsulated shared secrets are compared via `memcmp`; for ML-DSA, every generated signature is verified against the public key before the next iteration. All operations

passed these checks across every run. Additionally, the PQClean implementations we use pass PQClean’s own test harness (which includes NIST KAT vector comparisons) on the build host prior to cross-compilation.

D. Memory Profiling

Peak stack usage was measured using the stack-painting technique: the stack region is initialised with a sentinel value (0xDEADBEEF) before each operation, and the high-water mark is determined by scanning for the first overwritten word after execution. Flash footprint (text, data, BSS) was obtained using `arm-none-eabi-size -A` on the compiled ELF binary.

E. Cycle Counts

The RP2040’s Cortex-M0+ core lacks instruction and data caches, branch prediction, and speculative execution; all instructions execute in deterministic time relative to the system clock. We therefore derive cycle counts as $\text{cycles} = t(\mu\text{s}) \times 133$, where 133 is the core frequency in MHz. Tables II–IV report both wall-clock time and kilocycles (kc) to facilitate direct comparison with the pqm4 benchmarking framework [8], which reports Cortex-M4 cycle counts.

F. Energy Estimation

Energy per operation was estimated using the RP2040 datasheet power model: 3.3 V supply, 24 mA typical active-mode current at 133 MHz, yielding 79.2 mW active power [23]. Energy is calculated as $E(\mu\text{J}) = t(\mu\text{s}) \times 0.0792$. This constant-power model represents a conservative upper bound: the RP2040 draws less current during memory stalls and idle wait states than during sustained ALU computation, so actual energy consumption is likely lower than reported. We note that Halak et al. [9] employed the same datasheet-based estimation methodology for the RP2040 in their IEEE Access study. Because the power draw is assumed constant, energy is strictly proportional to execution time: the energy table (Table V) and Fig. 5 are a re-scaling of the timing results by a factor of $79.2 \mu\text{J ms}^{-1}$. We include them because energy budgets are a primary design constraint for battery-powered IoT devices, and expressing results in millijoules connects directly to battery capacity specifications. The *relative* rankings between algorithms remain valid regardless of the power model’s absolute accuracy, since all algorithms were measured on identical hardware at the same clock speed.

G. Statistical Measures

For each operation we report: mean, minimum, maximum, standard deviation, and coefficient of variation ($\text{CV} = \text{SD}/\text{mean} \times 100$). For ML-DSA signing, we additionally report the 95th and 99th percentiles to characterise worst-case latency for time-critical applications.

TABLE II
ML-KEM TIMING ON ARM CORTEX-M0+ (RP2040, 133 MHz). 30 RUNS PER OPERATION. $\text{CYCLES} = t(\mu\text{s}) \times 133$.

Variant	Op.	Mean (ms)	Cycles (kc)	Min (ms)	Max (ms)	CV
ML-KEM-512	KeyGen	9.94	1,322	9.89	10.68	1.45%
	Encaps	11.53	1,534	11.47	11.60	0.16%
	Decaps	14.23	1,893	14.12	14.28	0.19%
	<i>Total</i>	<i>35.71</i>	<i>4,749</i>			
ML-KEM-768	KeyGen	16.02	2,131	15.96	16.33	0.52%
	Encaps	18.58	2,471	18.49	18.62	0.11%
	Decaps	22.02	2,929	21.95	22.05	0.07%
	<i>Total</i>	<i>56.62</i>	<i>7,530</i>			
ML-KEM-1024	KeyGen	25.18	3,349	25.13	25.69	0.45%
	Encaps	28.10	3,737	28.07	28.15	0.06%
	Decaps	32.45	4,316	32.42	32.53	0.06%
	<i>Total</i> [†]	<i>85.73</i>	<i>11,402</i>			

[†]Total = keygen + encaps + decaps, measured sequentially on a single device (one party’s full handshake contribution).

H. Data Provenance

The results reported in this paper are from a single authoritative measurement campaign conducted after finalising the firmware build configuration. Preliminary runs (archived in the repository under `results/old_data/`) yielded consistent results within expected run-to-run variance ($<2\%$ for deterministic operations). Minor differences between preliminary and final runs—e.g., ML-KEM-512 full handshake of 36.3 ms in the preliminary run versus 35.7 ms in the final run—are attributable to a firmware rebuild with updated toolchain settings. The `reproduce.py compare` script in the repository enables statistical comparison between any two measurement runs.

V. RESULTS

A. ML-KEM Timing

Table II presents timing results for all three ML-KEM security levels. All operations exhibit low variance ($\text{CV} < 1.5\%$), confirming deterministic execution. With $n = 30$ runs and $\text{CV} < 1.5\%$, the 95% confidence interval for each mean is within $\pm 0.5\%$ of the reported value.

ML-KEM-512 completes key generation in 9.94 ms, encapsulation in 11.53 ms, and decapsulation in 14.23 ms, yielding a full key-exchange handshake (keygen + encaps + decaps) of 35.7 ms. Scaling to higher security levels, ML-KEM-768 requires 56.6 ms and ML-KEM-1024 requires 85.7 ms for a complete handshake.

B. ML-DSA Timing and Signing Variance

Table III presents ML-DSA timing results. Key generation and verification are deterministic ($\text{CV} < 1\%$). Signing, however, exhibits dramatic variance due to FIPS 204 rejection sampling.

ML-DSA-44 signing averages 158.9 ms but ranges from 70.1 ms (best case: signature accepted on first attempt) to 541.7 ms (worst case observed), with a coefficient of variation of 67.5% and a 99th-percentile latency of 489.9 ms. ML-DSA-65 averages 256.6 ms ($\text{CV} = 73.1\%$, $\text{p99} = 953.6$ ms), and ML-DSA-87 averages 355.2 ms ($\text{CV} = 65.6\%$, $\text{p99} = 1,125.0$ ms).

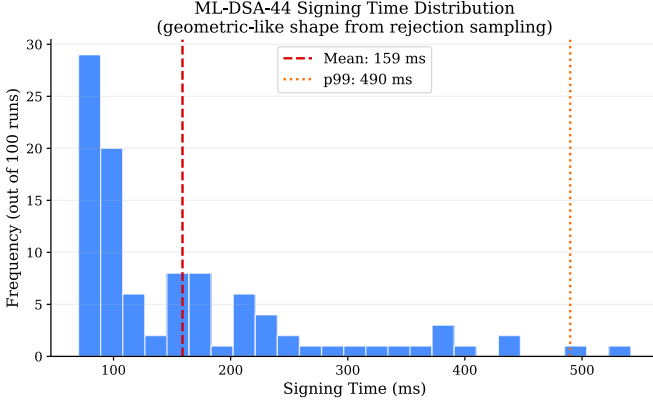


Fig. 1. ML-DSA-44 signing time distribution over 100 runs. The geometric-like shape reflects FIPS 204 rejection sampling: most signatures succeed within 1–2 iterations, but tail events exceed 500 ms.

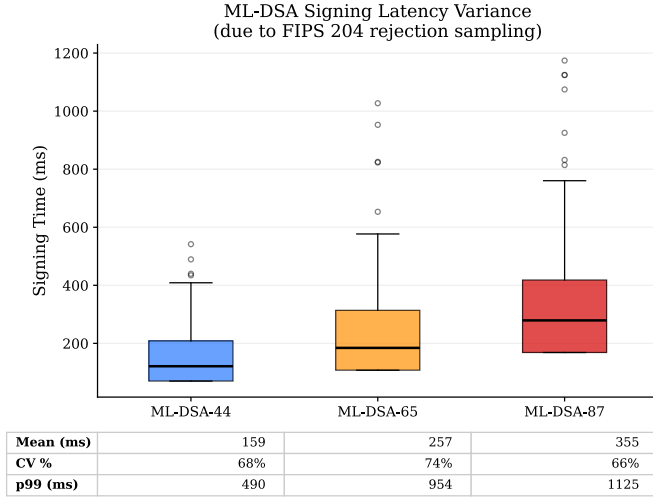


Fig. 2. ML-DSA signing latency variance across security levels. Box plots show the interquartile range; outliers represent high-iteration rejection sampling events.

This variance is an inherent property of the algorithm, not a measurement artefact: the geometric-like distribution of signing times (Fig. 1) matches the expected rejection-sampling behaviour specified in FIPS 204. Dividing the observed mean signing time by the expected iteration counts from the Dilithium specification [14] (approximately 4.25 for ML-DSA-44, 5.1 for ML-DSA-65, and 3.85 for ML-DSA-87; parameters unchanged in FIPS 204) yields per-iteration costs of roughly 37, 50, and 92 ms, respectively—consistent with the per-operation cost of the underlying NTT and matrix arithmetic on the Cortex-M0+.

C. Classical Baselines

Table IV presents classical algorithm timings on the same RP2040 hardware. RSA-2048 key generation is exceptionally slow (mean 215.6 s, individual runs ranging from 46.1 to 303.0 s) due to the probabilistic prime-finding process on a processor without hardware acceleration. Only $n = 5$ runs were collected (total wall-clock time ~ 18 min), yielding a wider confidence interval than the 30-run PQC benchmarks;

TABLE III
ML-DSA TIMING ON ARM CORTEX-M0+ (RP2040, 133 MHz). SIGNING: 100 RUNS; KEYGEN/VERIFY: 30 RUNS. CYCLES = $t(\mu s) \times 133$.

Variant	Op.	Mean (ms)	Cycles (kc)	CV	p95 (ms)	p99 (ms)
ML-DSA-44	KeyGen	39.8	5,297	0.9%	—	—
	Sign	158.9	21,135	67.5%	384.7	489.9
	Verify	44.0	5,849	0.1%	—	—
ML-DSA-65	KeyGen	68.4	9,100	0.1%	—	—
	Sign	256.6	34,130	73.1%	580.7	953.6
	Verify	72.2	9,604	0.0%	—	—
ML-DSA-87	KeyGen	114.3	15,201	0.5%	—	—
	Sign	355.2	47,246	65.6%	836.6	1125.0
	Verify	120.2	15,991	0.0%	—	—

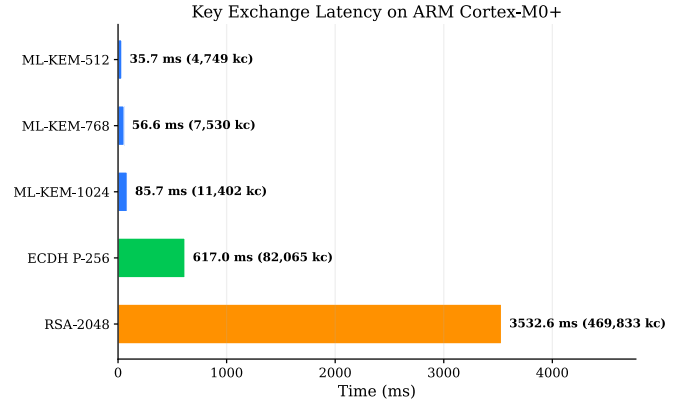


Fig. 3. Key exchange latency comparison on ARM Cortex-M0+. ML-KEM-512 achieves a full handshake (keygen + encaps + decaps) in 35.7 ms, $17\times$ faster than a complete ECDH P-256 key agreement (both parties' keygen and shared-secret computation).

the high CV of 43.4% reflects the inherent variability of prime search rather than measurement noise. RSA-2048 decryption requires 3,393 ms—two orders of magnitude slower than ML-KEM decapsulation. A complete ECDH P-256 key agreement—comprising both parties' key generation, public-value exchange, and shared-secret computation—takes 617.0 ms (82,061 kc). By contrast, a full ML-KEM-512 handshake (keygen + encaps + decaps) completes in 35.7 ms (4,749 kc), approximately $17\times$ faster for establishing a shared secret. We note that this comparison is between an IND-CCA2 KEM and a raw Diffie-Hellman key agreement; in practice, ECDH is typically wrapped in an authenticated key exchange (e.g., ECDH-ECDSA in TLS), which would add further overhead to the classical side.

D. Energy Consumption

Table V reports energy per key exchange and per signature cycle (keygen + sign + verify), estimated from the RP2040 datasheet power model (79.2 mW active).

A complete ML-KEM-512 key exchange consumes an estimated 2.83 mJ, versus 48.9 mJ for a full ECDH P-256 key agreement—a $17\times$ reduction. Even ML-KEM-1024 at 6.79 mJ remains $7.2\times$ more energy-efficient than ECDH. For signatures, ML-DSA-44 (keygen + sign + verify) averages 19.2 mJ versus 39.4 mJ for a full ECDSA P-256 cycle (keygen + sign + verify).

TABLE IV

CLASSICAL CRYPTOGRAPHY TIMING ON ARM CORTEX-M0+ (RP2040, 133 MHz). 30 RUNS PER OPERATION (RSA-2048 KEYGEN: 5 RUNS DUE TO ~ 216 S MEAN). CYCLES DERIVED AS $t(\mu s) \times 133$.

Algorithm	Op.	Mean (ms)	Cycles (kc)	CV
RSA-2048	KeyGen	215,612	28,676M	43.4%
	Encrypt	139.7	18,580	1.2%
	Decrypt	3,393	451,269	2.4%
ECDSA P-256	KeyGen	83.6	11,119	0.4%
	Sign	92.6	12,316	0.4%
	Verify	321.4	42,746	0.0%
ECDH P-256*	Key Agr.	617.0	82,061	0.4%

*Full handshake: `ecdh_make_params` (A keygen), `ecdh_make_public` (B keygen), `2×ecdh_calc_secret`.

TABLE V

ESTIMATED ENERGY PER CRYPTOGRAPHIC OPERATION ON RP2040 (DATASHEET MODEL: 3.3 V, 24 mA, 79.2 mW CONSTANT ACTIVE POWER).

Operation	Energy (mJ)
ML-KEM-512 handshake	2.83
ML-KEM-768 handshake	4.48
ML-KEM-1024 handshake	6.79
ML-DSA-44 sign cycle	19.22
ML-DSA-65 sign cycle	31.46
ML-DSA-87 sign cycle	46.71
ECDH P-256 key agr.	48.87
ECDSA P-256 sign cycle	39.41
RSA-2048 enc+dec	279.8

E. Memory Footprint

Table VI presents peak stack usage (measured via stack painting) and code size (from `arm-none-eabi-size`) for each algorithm variant.

ML-KEM-512's peak stack usage during decapsulation is 9.4 KB, fitting within a 10 KB allocation. ML-KEM-768 and -1024 require 14.2 KB and 20.0 KB respectively for decapsulation. ML-DSA is substantially more memory-intensive: ML-DSA-44 signing peaks at 50.6 KB, ML-DSA-65 at 77.6 KB, and ML-DSA-87 at 119.6 KB. The steep increase reflects the larger $K \times L$ polynomial vectors allocated on the stack by the PQClean reference C signing function. Our benchmarks use a custom linker script allocating 192 KB of stack to accommodate these requirements.

Code size scales modestly with security level for ML-KEM (5.1–6.7 KB), while ML-DSA requires 8.4–8.9 KB. Both families share a common PQClean utility library (~ 5 KB of application-relevant code for hashing and polynomial operations).

F. Cortex-M0+ vs Cortex-M4 Comparison

Table VII compares ML-KEM handshake times on our Cortex-M0+ results with published pqm4 Cortex-M4 *reference C* cycle counts from Kannwischer et al. [7] (pqm4 clean/reference implementations, not the optimised assembly variants; commit 3730b32a, compiled with `arm-none-eabi-gcc` at `-O3` on the STM32F4 Discovery

TABLE VI

MEMORY FOOTPRINT ON RP2040 (264 KB SRAM, 2 MB FLASH). PEAK STACK MEASURED VIA STACK PAINTING FOR THE WORST-CASE OPERATION. CODE SIZE FROM `ARM-NONE-EABI-SIZE`. TOTAL RAM = PEAK STACK + KEY MATERIAL + WORKING BUFFERS.

Variant	Peak Op	Stack (KB)	Code (KB)	Total RAM (KB)	Fits 264 KB
ML-KEM-512	decaps	9.4	5.1	12.1	✓
ML-KEM-768	decaps	14.2	5.2	17.7	✓
ML-KEM-1024	decaps	20.0	6.7	24.9	✓
ML-DSA-44	sign	50.6	8.9	55.5	✓
ML-DSA-65	sign	77.6	8.4	84.8	✓
ML-DSA-87	sign	119.6	8.7	128.9	✓

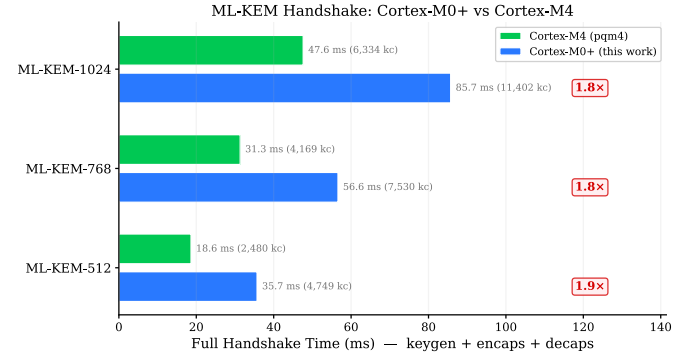


Fig. 4. ML-KEM handshake time: Cortex-M0+ (this work) vs Cortex-M4 (pqm4). The M0+ incurs a modest 1.8–1.9 $\times$ slowdown despite lacking UMULL, DSP, and SIMD instructions.

at 168 MHz). To enable a like-for-like comparison, the M4 millisecond column divides the published M4 *cycle counts* by 133 MHz (our M0+ clock), *not* by the M4's native 168 MHz. This normalisation shows how the M4 would perform at the same clock speed, isolating the microarchitectural difference. We note that the pqm4 optimised assembly implementations are 2–4 $\times$ faster than reference C on M4; the slowdown relative to *optimised* M4 code would therefore be proportionally larger (estimated 4–8 $\times$).

The Cortex-M0+ incurs a 1.80–1.92 $\times$ slowdown across security levels relative to M4 reference C. This is notably *lower* than the 3–7 $\times$ penalty sometimes assumed for M0+ versus M4, likely because: (i) the RP2040's single-cycle multiplier mitigates the absence of UMULL; (ii) PQClean reference C does not exploit M4-specific instructions, so the M4 advantage is reduced when both platforms run the same C code; and (iii) ML-KEM's NTT kernel is multiply-intensive, and the dominant cost is 32-bit multiplication, which both platforms execute in one cycle.

VI. DISCUSSION

A. Does ML-KEM Fit on Class-1 IoT Devices?

The RP2040's 264 KB SRAM comfortably accommodates all ML-KEM variants. Even the largest key object (ML-KEM-1024's 3,168-byte secret key) represents only 1.2% of available SRAM. However, stack requirements are non-trivial: ML-KEM-512 decapsulation peaks at 9,656 bytes (fitting a 10 KB

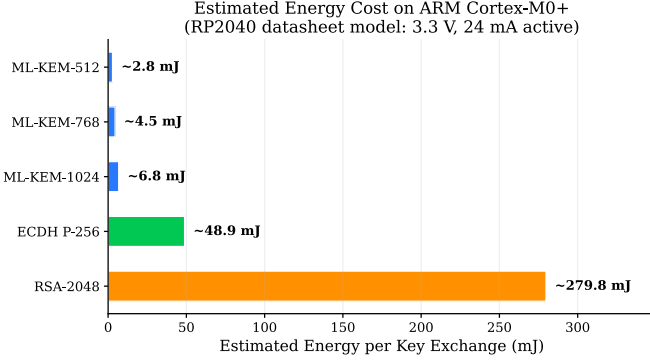


Fig. 5. Energy per key exchange on RP2040 (estimated from datasheet: 3.3 V, 24 mA). ML-KEM-512 consumes an estimated 2.83 mJ—94% less than ECDHP-256. Values are derived from the datasheet power model, not direct measurement.

TABLE VII
ML-KEM FULL HANDSHAKE: CORTEX-M0+ vs CORTEX-M4. M4 REFERENCE C CYCLE COUNTS FROM PQM4 [7] (COMMIT 3730B32A, -O3). M0+ COMPILED WITH -Os. M4 (MS) = M4 CYCLES / 133 MHz TO NORMALISE TO M0+ CLOCK SPEED.

Variant	M0+ (ms)	M0+ (kc)	M4 (ms)	M4 (kc)	Slow-down
ML-KEM-512	35.7	4,749	18.6	2,474	1.92×
ML-KEM-768	56.6	7,530	31.3	4,163	1.81×
ML-KEM-1024	85.7	11,402	47.6	6,331	1.80×

allocation), while ML-KEM-1024 requires 20,488 bytes. ML-DSA signing is far more demanding: ML-DSA-87’s signing function requires 119.6 KB of stack (Table VI), consuming roughly 45% of the RP2040’s total SRAM for the call stack alone. Deployments must ensure adequate stack allocation, particularly for ML-DSA at higher security levels. With a full handshake completing in 35.7 ms (ML-KEM-512) to 85.7 ms (ML-KEM-1024), ML-KEM is computationally feasible for Class-1 IoT key exchange at the algorithm level; protocol overhead (TLS/DTLS handshake, certificate chain) would add to the total latency in practice.

B. Is ML-DSA Signing Variance Acceptable?

The high coefficient of variation (66–73%) in ML-DSA signing is a significant concern for IoT applications with hard real-time constraints. The 99th-percentile signing time for ML-DSA-44 is 489.9 ms—nearly 7× the best-case time of 70.1 ms. For ML-DSA-87, the 99th-percentile reaches 1,125.0 ms (over 1 second).

For applications with 100 ms deadlines (e.g., industrial control loops), ML-DSA signing cannot guarantee timely completion even at the lowest security level. Systems deploying ML-DSA on Cortex-M0+ should implement either: (i) pre-computation strategies that sign during idle periods, or (ii) timeout-and-retry mechanisms with appropriate fallback behaviour. Alternatively, hash-based signatures (SLH-DSA/FIPS 205) offer deterministic signing times and merit investigation on this platform.

TABLE VIII
PQC DEPLOYMENT RECOMMENDATIONS BY IOT DEVICE CLASS (RFC 7228).

Class	Recommendation
Class 0 (<10 KB RAM)	Not feasible for software PQC; hardware accelerator required.
Class 1 (~100 KB RAM)	ML-KEM-512/768 recommended for key exchange. ML-DSA-44 feasible with variance-aware scheduling.
Class 2 (~250 KB+ RAM)	All ML-KEM and ML-DSA variants feasible. ML-KEM-1024 recommended where security margin permits.

C. Device-Class Compatibility

Based on our measurements, Table VIII provides deployment recommendations for IETF RFC 7228 device classes.

D. M0+ vs M4: Architectural Implications

The measured 1.8–1.9× slowdown is considerably lower than the 3–7× range sometimes cited in literature. This result must be contextualised: our comparison uses PQClean reference C compiled with -Os (optimise for size, Pico SDK default) on M0+ versus pqm4 reference C compiled with -O3 (optimise for speed) on M4. Since -Os typically produces slower code than -O3 for compute-bound workloads, the 1.8–1.9× slowdown includes a compiler-flag penalty in addition to the microarchitectural difference; with equal optimisation flags, the gap may be even smaller. The M4 advantage would increase significantly with assembly-optimised NTT implementations that exploit UMULL, barrel shifts, and SIMD halfword operations. Nevertheless, for the common case where IoT firmware uses vendor-provided C libraries without custom assembly, our results indicate that the M0+-to-M4 penalty is modest for ML-KEM.

E. RP2040-Specific Optimisation Opportunities

Several RP2040-specific features could further reduce PQC latency: (i) the second core could execute cryptographic operations while the primary core handles I/O; (ii) the hardware interpolator co-processors could accelerate address-generation patterns in the NTT; (iii) placing hot NTT loops in SRAM (via the `__not_in_flash_func` attribute) could eliminate QSPI flash access latency—the RP2040’s XIP cache (16 KB, 2-way set-associative) was active during all measurements, but NTT working sets may exceed this cache, causing flash wait states. These optimisations are left as future work.

F. Communication Overhead Considerations

For bandwidth-constrained IoT links, PQC object sizes matter as much as computation time. ML-KEM-512 requires transmitting an 800-byte public key and a 768-byte ciphertext (1,568 bytes total for a key exchange), compared to 65+65 bytes for ECDHP-256. This fits within a single CoAP/DTLS datagram (~1,280 bytes MTU) with fragmentation, but exceeds typical LoRa (222 bytes) and BLE (244 bytes) maximum payloads, requiring application-layer fragmentation or multiple radio

transmissions. ML-DSA-44 signatures (2,420 bytes) and public keys (1,312 bytes) are similarly challenging for constrained links. Hybrid classical+PQ modes (e.g., ECDH+ML-KEM as recommended by CNSA 2.0) would roughly double both the computation (~ 653 ms) and bandwidth requirements, though the PQ component dominates neither.

G. Limitations

This study has several limitations that should be acknowledged:

- **Single board tested:** all results are from one RP2040 unit. Manufacturing variation could affect absolute timings by a small margin.
- **Reference C only:** no ARMv6-M assembly optimisations were applied. Optimised implementations (e.g., the multi-moduli NTT approach of Li et al. [12]) would substantially reduce cycle counts.
- **No side-channel analysis:** timing leakage, power analysis, and electromagnetic emanation attacks were not evaluated. Bos et al. [11] have shown that masking Kyber on M0+ adds approximately $10\times$ overhead. Moreover, Wang et al. [29] demonstrated EM fault injection against ML-KEM and ML-DSA's Keccak permutation on Cortex-M0+ with 89.5% success rate, highlighting that unprotected implementations on this platform are vulnerable to physical attacks.
- **ROSC randomness:** the RP2040's ring oscillator is not cryptographically certified. Production deployments would require a DRBG seeded from a certified source.
- **No protocol overhead:** network protocol costs (TLS, DTLS, CoAP) are not included; our measurements reflect isolated algorithm execution only.
- **Energy estimation:** energy values are derived from a constant-power datasheet model, not direct current measurement. Saarinen's *ppps* measurements [33] on Cortex-M4 demonstrated over 50% variation in average power draw across different cryptographic primitives, driven by differing instruction mixes. Consequently, intra-family comparisons (e.g., ML-KEM-512 vs 768 vs 1024) are reliable since they share the same code structure, but cross-family comparisons (ML-KEM vs ECDH vs RSA) may carry larger error than the 10–20% suggested by the datasheet tolerance alone.
- **Stack measurement correction:** the v1 stack painting measurements for ML-DSA-65 and ML-DSA-87 were saturated by a 64 KB stack allocation. This revision uses 192 KB with overflow detection; corrected values are 77.6 KB and 119.6 KB respectively (Section VI-H).

H. Erratum: ML-DSA Stack Measurements

The initial version of this paper (arXiv v1) reported ML-DSA-65 and ML-DSA-87 peak stack usage of 54.4 KB and 51.6 KB respectively. These values were incorrect due to the stack painting region (64 KB) being smaller than the actual stack requirement. On bare-metal Cortex-M0+ without a hardware stack guard, the signing functions silently overflowed past the painted region, causing the measurement to saturate. This was

evidenced by two anomalies in the original data: (i) keygen, sign, and verify reported identical stack usage within each security level (impossible given their different code paths), and (ii) ML-DSA-87 reported *less* stack than ML-DSA-65 despite having strictly larger parameters ($K=8, L=7$ vs $K=6, L=5$). The corrected measurements with 192 KB stack allocation and overflow detection are: ML-DSA-65 sign = 77.6 KB, ML-DSA-87 sign = 119.6 KB (Table VI). ML-KEM and ML-DSA-44 values are unaffected. We thank Yaacov Belenky for identifying this issue.

VII. CONCLUSION

This paper presented the first systematic benchmarks of the NIST-standardised ML-KEM (FIPS 203) and ML-DSA (FIPS 204) on ARM Cortex-M0+, the most constrained 32-bit processor class widely deployed in IoT. On the RP2040 at 133 MHz, ML-KEM-512 completes a full key exchange in 35.7 ms at an estimated 2.83 mJ (datasheet power model)— $17\times$ faster than a complete ECDH P-256 key agreement. ML-DSA signing is feasible but exhibits 66–73% coefficient of variation due to rejection sampling, with 99th-percentile latencies reaching 1,125 ms for ML-DSA-87. The Cortex-M0+ incurs only a $1.8\text{--}1.9\times$ slowdown relative to published Cortex-M4 results when both run reference C code.

These results demonstrate that lattice-based PQC is practical on sub-\$5 IoT processors today, making the “harvest now, decrypt later” threat addressable even for the most constrained device class. We release all code, data, and analysis scripts as an open-source benchmark suite to enable reproducibility and community extension.

Future work includes: (i) ARMv6-M assembly optimisations for the NTT kernel; (ii) benchmarking SLH-DSA (FIPS 205) and FN-DSA (FIPS 206) on M0+; (iii) comparative evaluation on the RP2350 (Cortex-M33); (iv) integration with DTLS 1.3 and CoAP for end-to-end protocol measurements; and (v) side-channel resistance evaluation.

REFERENCES

- [1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [3] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [4] National Institute of Standards and Technology, “Stateless Hash-Based Digital Signature Standard,” FIPS 205, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/205/final>
- [5] T. Liu, G. Ramachandran, and R. Jurdak, “Post-quantum cryptography for Internet of Things: A survey on performance and optimization,” arXiv:2401.17538 [cs.CR], 2024. [Online]. Available: <https://arxiv.org/abs/2401.17538>
- [6] C. Bormann, M. Ersue, and A. Keranen, “Terminology for constrained-node networks,” RFC 7228, Internet Engineering Task Force, May 2014.
- [7] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4,” in *Proc. NIST 2nd PQC Standardization Conf.*, 2019.
- [8] M. J. Kannwischer, R. Krausz, J. Petri, and S. Yang, “pqm4: Benchmarking NIST additional post-quantum signature schemes on ARM Cortex-M4,” in *Proc. NIST 5th PQC Standardization Conf.*, 2024.

- [9] B. Halak, T. Gibson, M. Henley, C.-B. Botea, B. Heath, and S. Khan, "Evaluation of performance, energy, and computation costs of quantum-attack resilient encryption algorithms for embedded devices," *IEEE Access*, vol. 12, pp. 8791–8805, 2024. DOI: 10.1109/ACCESS.2024.3350775
- [10] A. Karmakar, J. M. Bermudo Mera, S. Sinha Roy, and I. Verbauwhede, "Saber on ARM: CCA-secure module lattice-based key encapsulation on ARM," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 3, pp. 243–266, 2018. DOI: 10.13154/tches.v2018.i3.243-266
- [11] J. W. Bos, M. Gourjon, J. Renes, T. Schneider, and C. van Vredendaal, "Masking Kyber: First- and higher-order implementations," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 4, pp. 173–214, 2021. DOI: 10.46586/tches.v2021.i4.173-214
- [12] L. Li, M. Wang, and W. Wang, "Implementing lattice-based PQC on resource-constrained processors: A case study for Kyber/Saber's polynomial multiplication on ARM Cortex-M0/M0+," in *Progress in Cryptology – INDOCRYPT 2023*, LNCS, vol. 14460, pp. 153–176, Springer, 2023. DOI: 10.1007/978-3-031-56235-8_8
- [13] J. W. Bos, J. Renes, and A. Sprengels, "Dilithium for memory constrained devices," in *Progress in Cryptology – AFRICACRYPT 2022*, LNCS, vol. 13503, pp. 217–235, Springer, 2022. DOI: 10.1007/978-3-031-17433-9_10 IACR ePrint: 2022/323.
- [14] L. Ducas *et al.*, "CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (version 3.1)," NIST PQC Round 3, 2021. Available: <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>
- [15] F. Abdulrahman *et al.*, "Taking ML-KEM & ML-DSA from Cortex-M4 to Cortex-M7 with SLOTHY," in *ACM ASIA CCS*, 2025.
- [16] J. Zhao *et al.*, "ESPM-D: Efficient sparse polynomial multiplication for Dilithium on ARM Cortex-M4 and Apple M2," arXiv:2404.12675, 2024.
- [17] H. Park, S. Seo, *et al.*, "Optimized implementation of CRYSTALS-Dilithium on 16-bit MSP430," *J. Inf. Security Appl.*, 2024.
- [18] F. Abdulrahman *et al.*, "pqmx: PQC for ARM Cortex-M55/M85 (Helium MVE)," GitHub/IACR, 2023.
- [19] J. Lopez, V. Cadena, and M. S. Rahman, "Evaluating post-quantum cryptographic algorithms on resource-constrained devices," in *IEEE QCE*, 2025. arXiv:2507.08312.
- [20] M. Grassl and L. Sturm, "Low-performance embedded IoT devices and the need for HW-accelerated PQC," in *IoTBDs*, SCITEPRESS, 2024.
- [21] H. Tschofenig, R. Housley, *et al.*, "Quantum-resistant security for software updates on low-power networked embedded devices," IACR ePrint 2021/577, 2021. arXiv:2106.05577.
- [22] P. Schwabe, "Post-quantum crypto on ARM Cortex-M," Microsoft Research Talk, 2019.
- [23] Raspberry Pi Ltd., "RP2040 Datasheet," 2024. [Online]. Available: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>
- [24] PQCclean Contributors, "PQCclean: Clean, portable, tested implementations of post-quantum cryptographic algorithms," GitHub, 2024. [Online]. Available: <https://github.com/PQCclean/PQCclean>
- [25] W. Ahmed, M. N. Bhutta, *et al.*, "Securing the future IoT with post-quantum cryptography," arXiv:2206.10473, 2022.
- [26] N. Ahmed, L. Zhang, and A. Gangopadhyay, "A survey of post-quantum cryptography support in cryptographic libraries," arXiv:2508.16078, 2025.
- [27] A. Hanna, A. Adebisola, *et al.*, "PQC-LEO: Evaluation framework for PQC on IoT networks," arXiv, Mar. 2026.
- [28] C.-M. M. Chung, V. Hwang, M. J. Kannwischer, G. Seiler, C.-J. Shih, and B.-Y. Yang, "NTT multiplication for NTT-unfriendly rings: New speed records for Saber and NTRU on Cortex-M4 and AVR," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 1, pp. 159–188, 2021.
- [29] Y. Wang, J. Yu, S. Qu, X. Zhang, X. Li, C. Zhang, and D. Gu, "Mind the faulty Keccak: A practical fault injection attack scheme apply to all phases of ML-KEM and ML-DSA," IACR ePrint 2024/1522, 2024.
- [30] PQShield Ltd., "PQMicroLib-Core: ML-KEM in 5 KB RAM for Cortex-M," Embedded World, Mar. 2026.
- [31] D. Dinu, "From ECDSA to ML-DSA: Migration analysis and implementation considerations," IACR ePrint 2025/2025, 2025.
- [32] Post-Quantum Cryptography Alliance (PQCA), "mlkem-native and mldsa-native: Production-ready PQC implementations," Linux Foundation, 2026. [Online]. Available: <https://github.com/pq-code-package>
- [33] M.-J. O. Saarinen, "Mobile energy requirements of the upcoming NIST post-quantum cryptography standards," *arXiv preprint arXiv:1912.00916*, 2019.

APPENDIX A BENCHMARK SOURCE CODE

The complete benchmark source code, raw data, analysis scripts, and reproduction instructions are available in the companion open-source repository:

<https://github.com/rojinc/pqc-cortex-m0-benchmark>

The repository includes:

- `src/bench_harness.h` — timing and stack measurement framework using the Pico SDK's `time_us_32()` function;
- `src/bench_mlkem.c` — ML-KEM benchmarks for all three security levels (30 runs each);
- `src/bench_mldsa.c` — ML-DSA benchmarks with 100-run signing for variance analysis;
- `src/bench_classical.c` — RSA-2048, ECDSA P-256, and ECDH P-256 baselines using mbedTLS 3.6.0;
- `scripts/add_energy.py` — datasheet-based energy estimation post-processor;
- `scripts/reproduce.py` — environment fingerprinting and cross-run statistical comparison;
- `results/raw/` — raw CSV benchmark data;
- `REPRODUCE.md` — step-by-step reproduction guide.